CloudOps Sentinel

Cloud infrastructure monitoring, reliability
and cost optimisation platform

Field	Detail
Domain	Cloud operations, SRE, observability, FinOps
Stack	Python 3.12 · FastAPI · Docker · Kubernetes · Prometheus · Grafana · GitHub Actions
Estate monitored	19 resources — 16 simulated Azure/AWS, 3 live Docker containers
Runs on	A laptop. No cloud account, no spend, no paid service.
Verified	69 unit tests · 50+ check end-to-end run against live containers
Headline result	$7,532/month estate · 55% waste identified · ~$55,000/year in findings

This guide is the complete reference: architecture and design rationale, the cloud/Docker/Kubernetes/CI-CD/monitoring/logging/cost/security material the project demonstrates, startup instructions, a scripted interview demonstration, two- and five-minute spoken explanations, 52 viva questions with answers, troubleshooting and deployment questions, resume bullets, and an honest account of what the project does not do.

Contents

1. What the project is 6

The constraint that shaped everything . . . 6

Verified, not asserted . . . 6

2. Architecture 7

The collection loop . . . 7

Design decisions and their trade-offs . . . 8

Data model . . . 8

3. Cloud concepts demonstrated 9

The shared responsibility model . . . 9

Azure and AWS equivalents . . . 9

Well-Architected pillars, and where each one appears . . . 9

The pricing models that drive the maths . . . 10

4. Docker 11

Multi-stage builds . . . 11

Compose: what the resource limits are actually for . . . 11

Reading a container's real resource usage . . . 12

5. Kubernetes 13

Why the control plane is a StatefulSet . . . 13

Three probes, three different questions . . . 13

Requests, limits and QoS . . . 13

Zero-downtime rollouts . . . 13

Configuration and rollout triggers . . . 14

Other manifest details worth defending . . . 14

6. CI/CD 15

What each stage actually enforces . . . 15

Supply chain and credentials . . . 15

7. Monitoring and observability 16

The metrics, and why these . . . 16

Health scoring . . . 16

Anomaly detection . . . 16

Alerting . . . 17
Prometheus and Grafana . . . 17

8. Logging 18

Structured, correlated, and level-disciplined . . . 18

9. Cost optimisation 19

The efficiency calculation . . . 19
The ten analysers . . . 19
Rightsizing that will not cause an incident . . . 19
Why posture findings sit in the cost report . . . 20
Result on the demo estate . . . 20

10. Security 21

No hardcoded credentials — and what that actually means . . . 21
RBAC and least privilege . . . 21
Network policy: default deny . . . 21
Container hardening . . . 22
Application hardening . . . 22
The deliberate exception . . . 22

11. Startup instructions 23

Prerequisites . . . 23
Start it . . . 23
Every command . . . 23
Configuration . . . 23
First-boot behaviour . . . 23

12. Interview demonstration 25

Minute 0-1: the estate . . . 25
Minute 1-2: cost . . . 25
Minute 2-5: the incident (the centrepiece) . . . 25
Minute 5-6: it is real engineering, not a demo script . . . 25
Minute 6-7: security and Kubernetes . . . 26

13. The two-minute explanation 27

14. The five-minute explanation 28

15. Viva questions and answers **30**

Architecture and design . . . 30
Cloud . . . 31
Docker . . . 32
Kubernetes . . . 33
CI/CD . . . 35
Monitoring, logging and detection . . . 36
Cost optimisation . . . 38
Security . . . 39
Python and implementation . . . 40

16. Troubleshooting questions **43**

17. Deployment questions **45**

18. Resume bullets **47**

Full version . . . 47
Compact version (three lines) . . . 47
Skills this evidences . . . 47

19. Limitations **49**

What I would build next, in order . . . 49
Closing note . . . 50

1. What the project is

CloudOps Sentinel is a cloud operations platform that answers three questions from one telemetry pipeline: **is the estate working**, **is it going to keep working**, and **why does it cost this much**. In most organisations those are three separate tools owned by three different teams, and they disagree with each other — which is how an argument about whose number is right replaces fixing the problem.

Joining them is not a cosmetic choice. Utilisation is a reliability signal and it is simultaneously the numerator of the cost-efficiency calculation. A database at 9% CPU is healthy, is a capacity question, and is $2,715 a month of waste — all at once. A billing export can never tell you the third thing, because the invoice shows cost and never shows waste.

The constraint that shaped everything

The requirement was that it runs on a laptop with no cloud spend. The lazy answer is to mock everything, which produces a demo that proves nothing because the detection logic is handed the answers. The approach taken here splits the estate in two.

	Simulated half (16 resources)	Live half (3 containers)
Source	config/inventory.yaml	Real Docker containers
Telemetry	Deterministic generator	cgroup CPU/memory, scraped over HTTP
Chaos	Metric modifiers	Genuinely burns CPU, leaks memory, returns 500s, exits and is restarted
Provides	Breadth: RDS, S3, Lambda, load balancers, orphaned disks, two clouds, real SKUs, regions, tags	Truth: the detection path is never told an incident was requested

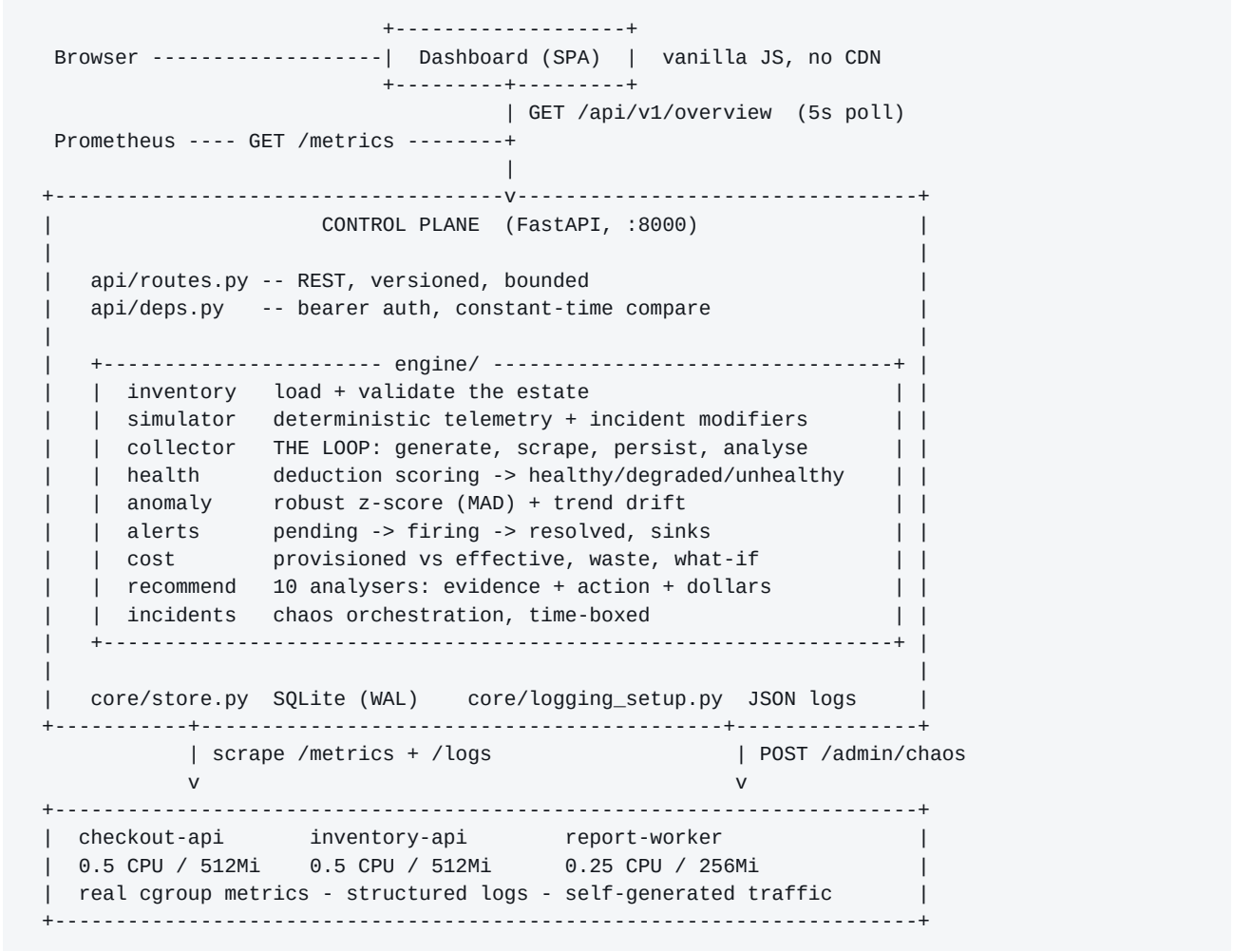
Everything above the collector treats both identically. The health scorer, anomaly detector, alert engine and cost model only ever see a `Resource` and a stream of samples — they cannot tell which half a number came from. That is what makes the demonstration honest: when a container is pushed to 94% CPU, the alert that fires was worked out from measurements arriving over the network.

Verified, not asserted

All end-to-end checks pass (53/53 on the run below) against the running stack, including injecting real chaos into a real container and asserting the pipeline reacts unaided. 69 unit tests cover the analysis engines. The same script CI runs is the script a developer runs — there is no CI-only path that can pass while the real thing is broken.

```
7. Incident simulation -> detection (live container)
  PASS  scenario catalogue available  9 scenarios
  PASS  incident accepted
  PASS  chaos injected into the real container
  ...waiting up to 110s for the pipeline to react on its own
  PASS  container CPU actually rose  peak 91.5%
  PASS  alert rule fired from the measurements alone
  PASS  health score degraded  score 83
================================================================
ALL CHECKS PASSED  53/53
```

2. Architecture



The collection loop

One tick every `COLLECT_INTERVAL_SECONDS` (default 10):

1. **Expire incidents.** Anything past its end time closes automatically, so a demo cannot leave the fleet wedged.
2. **Generate** samples for simulated resources, applying active incident modifiers.
3. **Scrape** live containers concurrently with `asyncio.gather`.
4. **Persist** samples and logs in two batched writes.
5. **Analyse:** score health, run both anomaly detectors, evaluate alert rules.
6. **Publish** every value as a Prometheus gauge.
7. **Prune** past the retention window, roughly once a minute.

> **A failed scrape is data, not a gap.** It records `availability=0` rather than skipping the resource. Skipping would make a dead target indistinguishable from a healthy one that simply did not match a rule — and "down" would become unalertable.

The whole loop is wrapped in a broad exception handler that logs and continues. A monitoring system whose collector dies on one malformed sample is worse than no monitoring, because everyone assumes silence means health.

Design decisions and their trade-offs

Choice	Alternative	Why
Pull (scrape)	Push	A target that dies is detected by its *absence*. With push, a silent target and a healthy target look identical.
SQLite (WAL)	Postgres/Timescale	Must run on a laptop. Few hundred rows per tick. Append-only series + small mutable tables ports upward unchanged. Cost: single writer.
Robust z-score (median + MAD)	Mean + stdev, or ML	Explainable at 3am, no training job, cold-start safe. One outlier inflates a stdev enough to hide the next real anomaly.
Deterministic simulator	random per sample	value(resource, metric, t) is a pure function, so backfill is continuous with live data and the demo reproduces exactly. White noise would make any detector look good.
Rule-based recommendations	ML sizing model	Every finding shows its evidence. "The model said so" does not survive review by the team that owns the resource.
Vanilla-JS dashboard	React + chart library	No build step, no CDN, no npm dependency tree bolted onto a security tool. Works air-gapped.
No Docker socket	Mount /var/run/docker.sock	That socket is root on the host. A restart counter is not worth it — start-time inference is used instead.

Data model

```
samples(ts, resource_id, metric, value)       -- append-only time series
logs(ts, resource_id, service, level, message, context)
alerts(fingerprint PK, rule, resource_id, status, value, first_seen, ...)
alert_history(ts, fingerprint, event, ...)    -- audit trail
anomalies(ts, resource_id, metric, value, baseline, score, method, ...)
incidents(id PK, scenario, resource_id, started_at, ends_at, status, ...)
```

Alert state lives in the database, not in memory, so it survives a restart of the control plane — an alerting system that forgets everything when redeployed will forget it during the deploy that caused the incident. The fingerprint is `rule:resource_id`, stable, so one ongoing problem updates one row instead of accumulating thousands.

3. Cloud concepts demonstrated

The shared responsibility model

The provider secures *of* the cloud (hypervisor, physical hosts, managed service internals); you secure *in* the cloud (identity, network exposure, encryption settings, patching your images, your data). Every configuration finding this project raises — a public database, an unencrypted disk, anonymous blob access, static credentials — sits squarely on the customer side. The provider will happily let you make all of them.

Azure and AWS equivalents

Concept	Azure	AWS	In this project
Account boundary	Subscription	Account	`subscriptions:` in inventory.yaml
Grouping	Resource group	Tags / resource groups	`resource_group`
VM	Virtual Machine (D8s_v5)	EC2 (m6i.2xlarge)	virtual_machine
Managed K8s	AKS	EKS	kubernetes_node (+ control-plane fee)
Serverless	Functions (Consumption)	Lambda	serverless_function, free tier modelled
Managed SQL	Azure SQL (GP_Gen5_8)	RDS (db.r6g.4xlarge)	managed_database, 1.15x premium
Object storage	Blob Storage	S3	object_storage, hot/cool/archive
Load balancing	Load Balancer / App Gateway	ALB / NLB	load_balancer, fixed + per-GB
Workload identity	Managed Identity	IAM Role / IRSA	`managed_identity` posture flag
Secret store	Key Vault	Secrets Manager	Recommended over K8s Secrets
Monitoring	Azure Monitor	CloudWatch	The collector abstraction
Cost data	Cost Management API	Cost & Usage Report	config/pricing.yaml
Advisor	Azure Advisor	Trusted Advisor / Compute Optimizer	The recommendation engine

Well-Architected pillars, and where each one appears

- **Operational excellence** — structured logging, correlation IDs, a runbook section per alert rule, infrastructure as code, CI that verifies before it publishes.
- **Security** — no hardcoded credentials, least-privilege RBAC, default-deny networking, non-root containers, posture auditing. See section 9.
- **Reliability** — health probes distinguishing liveness from readiness, restart tracking, anomaly detection, alerting with *for* durations, PDBs, topology spread.
- **Performance efficiency** — rightsizing from measured p95, HPA on CPU and memory, resource requests that make scheduling and autoscaling possible at all.
- **Cost optimisation** — the entire cost engine: waste quantification, rightsizing, orphan detection, scheduling, tiering, commitment analysis.

The pricing models that drive the maths

- **Allocation-billed** (VMs, databases, disks): you pay for provisioned capacity whether you use it or not. This is where waste lives and where rightsizing pays.
- **Consumption-billed** (Lambda/Functions, S3/Blob requests): you pay for what runs. You cannot over-provision it; the lever is memory sizing and tiering instead.
- **Commitment discounts** (Reserved Instances, Savings Plans): ~28% for one year, ~46% for three. **Rightsize first** — committing to an oversized instance locks in the waste for a year while the invoice shows a saving.
- **Spot / low-priority**: ~70% off for interruptible workloads. The discount is real; the interruption rate is not modelled here, so treat it as an upper bound.

4. Docker

Multi-stage builds

```
FROM python:3.12-slim-bookworm AS builder
COPY requirements.txt .                     # deps BEFORE source: cache reuse
RUN python -m venv /opt/venv \
 && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim-bookworm AS runtime
RUN groupadd --system --gid 10001 cloudops \
 && useradd --system --uid 10001 --gid cloudops --no-create-home \
            --shell /usr/sbin/nologin cloudops
COPY --from=builder /opt/venv /opt/venv     # only the venv ships
COPY --chown=cloudops:cloudops app ./app
USER 10001:10001
HEALTHCHECK --interval=15s --timeout=4s --start-period=45s --retries=3 \
    CMD python -c "import urllib.request,sys; sys.exit(0 if ...)"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- **Two stages.** Build tools, pip caches and package indexes never reach the shipped image. Smaller, and an entire class of tools is unavailable to anyone who gets a shell.
- **Dependencies copied before source.** Editing a .py file reuses the cached dependency layer instead of reinstalling everything. Single biggest lever on rebuild time in CI.
- **Non-root, fixed UID 10001.** Root in a container means a container-escape bug is a root-on-host bug. A fixed non-zero UID also lets Kubernetes pin `runAsUser`.
- **HEALTHCHECK using stdlib urllib** — no curl needs to be in the image. `--start-period` covers the first-boot backfill so a slow start is not read as a broken container.
- **Exec-form CMD.** uvicorn becomes PID 1 and receives SIGTERM directly, so the shutdown hook runs. Shell form leaves /bin/sh as PID 1 swallowing the signal until Docker kills the process outright.

Compose: what the resource limits are actually for

```
x-demo-defaults: &demo-defaults          # YAML anchor: identical hardening
  security_opt: [no-new-privileges:true]
  cap_drop: [ALL]
  logging:
    driver: json-file
    options: {max-size: "10m", max-file: "3"}

checkout-api:
  <<: *demo-defaults
  deploy:
    resources:
      limits: {cpus: "0.50", memory: 512M}
      reservations: {memory: 128M}
```

The CPU limit is not decoration. The demo service reports utilisation as a percentage of *this envelope*, read from its own cgroup — so saturating half a core reads as 100%, exactly as Kubernetes would compute it against a pod limit. That distinction is why a container can be throttled to a standstill on a host that looks completely idle.

- `depends_on: condition: service_healthy` — the control plane waits for real health, not merely for a container to be created.
- `./config:/app/config:ro` — the app has no business rewriting its own inventory or rate card at runtime.

- Log rotation capped at 10 MB x 3, or a chatty container fills the host disk.
- The Grafana password arrives as a **Docker secret from a file**, not an environment variable — env vars are visible in `docker inspect` to anyone on the host.
- `profiles: [observability]` keeps Prometheus and Grafana optional, so the core stack starts fast.

Reading a container's real resource usage

`ResourceProbe` reads the same source `docker stats` and the kubelet use, falling back cgroup v2 → cgroup v1 → `/proc`:

```
/sys/fs/cgroup/cpu.max        ->  "50000 100000"  = 0.5 CPU limit
/sys/fs/cgroup/cpu.stat       ->  usage_usec (cumulative)
/sys/fs/cgroup/memory.current ->  bytes in use
/sys/fs/cgroup/memory.max     ->  the limit

cpu% = 100 * (delta_usage_seconds) / (elapsed * cpu_limit)
```

CPU is a rate, so it needs two readings and the elapsed time between them; the result is smoothed with an EWMA because a single 200 ms window is far too twitchy to alert on.

5. Kubernetes

Why the control plane is a StatefulSet

It owns a SQLite file. A Deployment with a PVC would let a rolling update run two pods against the same file, and SQLite does not survive that. The StatefulSet ordering guarantee — old pod fully terminated before the new one starts — plus a single replica is what makes single-writer storage safe. Scaling past one replica means moving to Postgres first, which is recorded in the limitations rather than pretended away. The stateless demo services are Deployments, which is exactly why they can have an HPA and the control plane cannot.

Three probes, three different questions

Probe	Question	Failure action	Depends on downstream?
startupProbe	Has it finished booting?	Keep waiting (30 x 5s here)	No
livenessProbe	Is the process wedged?	Kill and restart the pod	**Never**
readinessProbe	Can it serve right now?	Remove from the Service endpoints	Yes, legitimately

The single most important probe rule: liveness must never depend on a downstream service. If it does, one slow dependency makes every pod fail liveness at once and the cluster restarts the entire fleet — turning a partial outage into a total one. Note that in this project the demo service keeps `/healthz` returning 200 during an induced outage while `/readyz` returns 503: restarting is the wrong remedy for a dependency being down; removing from the load balancer is the right one.

A **startupProbe** is the correct fix for a slow-starting application, not a slacker liveness probe. It grants a long grace period once, then hands over to a tight liveness probe for the rest of the pod's life. The control plane uses 30 x 5s to cover its first-boot backfill.

Requests, limits and QoS

- **Requests** are what the scheduler reserves — they decide placement. **Limits** are the ceiling: exceeding CPU means throttling, exceeding memory means an OOM kill.
- **The HPA divides usage by the request.** A pod with no request has no denominator, so the HPA silently does nothing. This is the most common reason "autoscaling is broken".
- **QoS classes:** Guaranteed (requests == limits) is evicted last; Burstable next; BestEffort (no resources at all) is evicted first. The control plane sets memory request == limit so the component that has to survive to explain the outage is the last one evicted.
- A **LimitRange** gives every container a default, so a pod authored without resources cannot land as BestEffort by accident. A **ResourceQuota** bounds the namespace so a runaway HPA cannot consume the cluster.

Zero-downtime rollouts

```
strategy:
  rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
lifecycle:
  preStop:
    exec: {command: ["sleep", "5"]}
```

`maxUnavailable: 0` never drops below current capacity. The `preStop` sleep is the subtle one: endpoint removal and container termination happen *concurrently*, so without a delay the container stops accepting connections while kube-proxy is still routing to it. Omitting this is the most common cause of 502s during an

otherwise correct rolling update.

Configuration and rollout triggers

The kustomize `configMapGenerator` reads the *same* `config/*.yaml` files that docker-compose mounts, so there is exactly one source of truth. The generated name carries a content hash, so editing the inventory produces a new ConfigMap name, which changes the pod spec, which triggers a rollout automatically. That is the built-in answer to "I changed the config and nothing happened" — a mounted ConfigMap edit is otherwise invisible to running containers.

Other manifest details worth defending

- **Pod Security Admission** at `restricted`, with `warn` and `audit` alongside `enforce` so a violation is visible rather than silently rejected.
- **topologySpreadConstraints** with `ScheduleAnyway`, not `DoNotSchedule` — a hard constraint would leave the second replica Pending forever on a single-node kind or minikube cluster.
- **PodDisruptionBudget** on the control plane is `maxUnavailable: 1`. With one replica a PDB cannot preserve availability anyway; `0` would make `kubectl drain` hang forever and block node upgrades — a worse failure than a 20-second gap.
- **report-worker deliberately ships without a liveness probe.** It is the live example behind the "missing health check" recommendation, which the engine flags on every run.
- **Prometheus annotations** rather than a ServiceMonitor CRD, so this works on a bare cluster as well as one running the operator.

6. CI/CD

```
lint  ->  test  ->  build  ->  e2e       ->  security  ->  publish  ->  deploy
                         \-> k8s-validate               (main only)   (gated)
```

Stages are ordered by **how fast a stage can tell you that you are wrong**. Lint and unit tests take seconds and run first; the container build takes a minute; the end-to-end stage brings the whole stack up. Nothing downstream runs if something upstream failed.

What each stage actually enforces

Stage	Enforces
lint	ruff lint + format; yamllint; and a config check that every resource type has a pricing entry (otherwise it silently costs $0 and vanishes from the FinOps view) and every alert rule has a runbook.
test	69 unit tests with coverage; results uploaded even on failure — a red build with no visible reason is the worst CI experience.
build	Both images build; **fails if either runs as root**; fails if either lacks a HEALTHCHECK; layer caching via type=gha.
e2e	Boots the full stack, runs the same scripts/verify_local.py a developer runs — including injecting real chaos and asserting detection. Then checks the Prometheus exposition and that every log line is valid JSON with the required fields.
k8s-validate	kustomize build, kubeconform schema validation, then Python assertions on the rendered YAML: runAsNonRoot, no default ServiceAccount, allowPrivilegeEscalation false, readOnlyRootFilesystem, drop ALL, requests+limits present, readiness probe present, no RBAC wildcards, no access to secrets, no literal credential in any env var, restricted PSA.
security	Trivy on images and filesystem, SARIF to the security tab. **Secret scanning is a hard failure; base-image CVEs are not.**
publish	GHCR via job-scoped GITHUB_TOKEN, with SBOM and provenance. main branch pushes only — a fork PR can never push an image.

The asymmetry in the security stage is deliberate. A committed credential is always actionable, so it fails the build. An unfixable base-image CVE is not, and failing on it produces a permanently red pipeline that everyone learns to ignore — and then they ignore the real one too. Unactionable alerts destroy the value of actionable ones.

Supply chain and credentials

- `permissions: contents: read` at the top; each job opts into more. The publish job alone gets `packages: write`.
- **No long-lived PAT.** The ephemeral job-scoped `GITHUB_TOKEN` is used, so there is nothing to rotate or leak.
- `concurrency` with `cancel-in-progress` — a new push supersedes an obsolete run.
- Images published with **SBOM and build provenance**, so a running image traces back to the exact commit and workflow run.
- Deployment is a **gated environment**, which is what gives a required reviewer and an audit trail. The real steps would authenticate by OIDC federation (azure/login with a federated credential, or configure-aws-credentials with role-to-assume) — **no static cloud key in a secret**.

7. Monitoring and observability

The metrics, and why these

The four **golden signals** (latency, traffic, errors, saturation) plus the container-specific ones and the cost dimension:

Metric	Type	Signal	Why it earns its place
cpu_utilization	gauge %	Saturation	Drives rightsizing and the HPA; % of the container limit, not the host
memory_utilization	gauge %	Saturation	Predicts OOM kills — the failure that looks like a crash, not a slowdown
disk_utilization	gauge %	Saturation	A full volume fails writes with errors that name nothing useful
latency_p95_ms	gauge ms	Latency	p95, not mean — an average hides the tail users actually experience
error_rate	ratio	Errors	Computed from counter deltas, so it is a rate not a lifetime average
requests_per_second	gauge	Traffic	Distinguishes "load" from "fault" when CPU rises
restart_count	counter	Reliability	Crash loops; must be alerted with increase(), never absolutely
availability	gauge 0/1	Reliability	A failed scrape records 0 — down is a measurement
estimated_cost / waste	gauge USD	Cost	Makes spend a first-class monitored signal, not a monthly surprise

Health scoring

A deduction model: start at 100, subtract for each fault, list the reasons. Chosen because it is **auditable** — anyone can reconstruct the number from the reasons, which is not true of a weighted average. Failing probe –60, error rate above 5% –30, CPU above 95% –25, and so on. Only the most severe band per metric counts, so 97% CPU is not charged twice for also being above 85%. Status: ≥85 healthy, ≥60 degraded, below that unhealthy.

Stale telemetry costs 40 points and is called out explicitly, because **no data is a monitoring failure, not a pass**. Separately, a resource type with no applicable metrics — an unattached elastic IP emits nothing, ever — is scored `not_monitored` rather than `unknown`, so it does not drag the fleet status down. That distinction was a real bug found during verification.

Anomaly detection

```
median  = median(window)
mad     = median(|v - median|)          # NOT standard deviation
scale   = mad * 1.4826                  # consistent estimator of sigma
z       = (current - median) / scale
```

Why median and MAD rather than mean and standard deviation: a single enormous outlier inflates a standard deviation so much that the next genuine anomaly falls inside it — the detector goes blind exactly when it matters most. A test asserts this: with a history of fifty-nine 40s and one 5000, a value of 95 is still detected.

Two refinements, both of which came out of test failures rather than theory:

- **An absolute-delta floor per metric.** On a perfectly flat series MAD is zero and any z-score explodes, so a healthy 0.2% error rate moving to 0.4% would be reported as an infinite-sigma event. When MAD collapses, the fallback is the metric's domain-significance floor — *not* the standard deviation, which would reintroduce exactly the non-robustness MAD exists to avoid.
- **A second detector for slow ramps.** A memory leak defeats a z-score structurally: the baseline drifts upward with the value, so by the time it is at 95% the median is at 60% and the spread is enormous. The trend detector compares the recent window against the preceding one, normalised by the *recent* window's own spread — a trending series is locally quiet and globally wide, so local noise is the honest denominator.

Detections are deduplicated per (resource, metric) over a cooldown, so one sustained incident produces one anomaly record rather than one per scrape.

Alerting

Prometheus semantics, because they are the ones that survived contact with real on-call rotations:

- **pending → firing.** A rule must hold continuously for its `for` duration before it pages. This single mechanism kills most false pages: one bad scrape is not an incident.
- **firing → resolved.** The condition must stay clear for `resolve_after_seconds`, so a flapping target does not generate a fresh page every 30 seconds.
- **Notification only on the pending→firing edge**, never every tick. An alert that re-notifies continuously trains people to ignore it.
- **Stable fingerprints** and state in SQLite, so alert state survives a redeploy.
- **A broken sink cannot break the loop** — sinks are called in a try/except, because a dead webhook must never take down the collection that produced the alert.
- **Every rule links to a runbook section.** CI fails the build if one does not.

Never alert on a raw counter. `restart_count` only ever increases, so `restart_count > 2` latches on the first crash loop and can never resolve. The rule uses windowed `increase` aggregation — the direct equivalent of `increase(restart_count[10m]) > 2` in PromQL. This was a genuine bug caught during verification.

Prometheus and Grafana

The control plane is an **exporter**: it re-publishes the whole simulated estate as `cloudops_resource_*` series, which is the standard pattern for anything Prometheus cannot scrape directly — a cloud billing API, a SaaS, a legacy appliance. The demo services are scraped directly as ordinary instrumented apps. The result is that the same PromQL works across both halves.

Label cardinality is the discipline that keeps this from falling over. HTTP metrics are labelled by *route template* (`/inventory/{resource_id}`), never by raw path — the raw path would mint a new time series per resource ID, which is the classic way to take Prometheus down.

Grafana's datasource and dashboard are **provisioned from files** with a pinned datasource UID and `allowUiUpdates: false`. A hand-clicked Grafana is a snowflake: it cannot be recreated, reviewed or rolled back, and the person who set it up becomes a dependency.

The platform also **monitors itself**: `CollectorStalled` fires when no collection has completed for two minutes. The failure mode of monitoring is silence, and silence is indistinguishable from health.

8. Logging


```
{"timestamp":"2026-09-03T17:13:30.873Z","level":"WARNING","logger":"cloudops.alerts",
 "message":"alert fired: CPU above 85% on checkout-api",
 "service":"cloudops-control-plane","environment":"local","version":"1.0.0",
 "correlation_id":"03b40f9b90884cdb","event_type":"alert",
 "rule":"HighCpuUtilization","resource_id":"svc-checkout-api",
 "severity":"warning","value":99.51,"threshold":85.0,
 "runbook":"docs/RUNBOOK.md#high-cpu"}
```


One rule: JSON to stdout, nothing else. No log files, no rotation, no shipper compiled into the application. The container writes to stdout; the runtime captures it; something downstream forwards it. That is the twelve-factor contract, and it is what makes the application portable across Docker's json-file driver, a Fluent Bit DaemonSet, Azure Container Insights, CloudWatch, or Promtail/Loki without knowing which one it is running under.

An application that writes log files inside a container creates three problems at once: the files vanish when the container is replaced, they fill the writable layer, and they are invisible to platform log collection. The Kubernetes manifests mount a **read-only root filesystem**, which makes it impossible by construction rather than by convention.

Structured, correlated, and level-disciplined

- **Queryable immediately.** `{app="cloudops"} | json | severity="critical"` works whatever the message text says, because the fields are fields. A formatted string needs a regex that breaks when someone rewords the message.
- **Correlation IDs in a ContextVar** — the async-safe equivalent of thread-local storage. With hundreds of coroutines interleaving on one thread, a module-level variable would leak one request's ID into another's lines. The ID is echoed in the response header, so a user reporting a failure hands you the exact key.
- **event_type as a discriminator**: http_access, alert, anomaly, incident_start.
- **Not logging INFO per sample.** 19 resources at 10s would be 164,000 lines a day of noise in which the one line that mattered is invisible. Lines are emitted when something is notable, plus a five-minute heartbeat.
- **A bounded deque (maxlen=500)** for the pull-able buffer. An unbounded buffer in a container with a 512 MiB limit is an OOM kill waiting for a busy afternoon — and the crash would look like an application bug.

The access log is emitted once, by the middleware that knows the correlation ID and route template; uvicorn's own access logger is disabled outright. Leaving both on doubles the volume and hands the shipper **two different schemas for the same event** that disagree with each other. This was a real bug: the logging setup ran during the FastAPI lifespan, after uvicorn had configured its own, and was re-enabling propagation on a logger that `--no-access-log` had already suppressed.

CI parses the last 200 log lines from the running container and fails the build if any is not valid JSON or is missing a required field. Structured logging is a contract: if one line is unparseable the shipper drops it, and the evidence is gone exactly when an incident needs it.

9. Cost optimisation

You are billed for what you provisioned, not for what you used. A 16-vCPU database at 9% CPU costs exactly the same as one at 90%. That gap is the whole subject, and it is invisible to a billing export — the invoice shows cost and never shows waste. Utilisation lives in the monitoring system, price lives in the billing system, and the number that matters only exists when you join them.

The efficiency calculation

```
util       = max(cpu_p95, memory_p95)
efficiency = min(util / 80.0, 1.0)
waste      = monthly_cost * (1 - efficiency)
```

- **max, not average.** A box at 80% memory and 5% CPU is not 42% efficient — it is 80% efficient, because you cannot shrink past the dimension that is full. Averaging would recommend halving a box that would immediately start OOM-killing.
- **Divide by 80, not 100.** Headroom is not waste. A resource at 80% is correctly sized — you need room for a spike, a failover, a GC pause. Targeting 100% would flag every well-run service as 20% wasteful, and the report would be dismissed by the people who most need to read it.

The ten analysers

Analyser	Trigger	Action	Saving
over_provisioned	p95 CPU 10-40%, 8+ samples	Resize down the SKU ladder	40-60%
under_utilised	p95 CPU <10% and memory <20%	Decommission, or smallest SKU / spot	75%
orphaned_resource	Disk or IP with attached: false	Snapshot, then delete	100%
storage_tiering	>500 GB hot, no lifecycle policy	Cool at 30d, archive at 180d	~45% of cold data
scheduling	Non-prod compute >$20/mo	Start/stop 08:00-20:00 Mon-Fri	64%
commitment_discount	Prod, p95 CPU >=40%, >$50/mo	1-year RI / Savings Plan	28%
unhealthy_container	3+ restarts or availability <95%	Exit code, OOM status, probe thresholds	-
high_error_rate	p95 error rate >=3%	Correlate with deploys; roll back first	-
suspicious_configuration	Posture flags	Per-issue remediation	-
missing_health_check	health_check: false	Add liveness + readiness probes	-

Rightsizing that will not cause an incident

```
needed_vcpu   = current_vcpu * (cpu_p95 / 60)     # target 60% AFTER the resize
proposed_vcpu = next rung DOWN the ladder covering needed_vcpu
proposed_mem  = max(proportional, memory_p95 * 1.25)   # never below peak
```

1. **Snap to a real SKU.** "6.4 vCPU" is useless because no cloud sells it. Round down the ladder 0.25, 0.5, 1, 2, 4, 8, 16, 32, 64.
2. **Target 60%, not 100%.** Sizing to the observed peak leaves zero headroom, and the first spike after the change becomes an incident blamed on the cost tool — correctly.
3. **Memory never shrinks below observed peak + 25%.** CPU pressure makes a service slow; memory pressure makes it die. Not symmetric risks, not treated symmetrically.

4 **It refuses to guess.** Under 8 samples the sizing analysers stay silent. Confidently telling someone to halve a production database off four data points is how a cost tool loses credibility permanently — and it only gets one chance.

Rightsize before you commit. The commitment analyser skips anything under 40% p95 CPU and its action text says so explicitly. Buying a one-year reservation for an oversized instance locks in the waste for a year, while the invoice shows a saving. For the same reason, `risk` is medium on commitments and high on decommissioning — a contractual lock-in and an irreversible delete are not low-risk just because they save money.

Why posture findings sit in the cost report

The resources nobody owns are the ones that are both insecure *and* expensive — the untagged, unencrypted, unmonitored jumpbox from 2023 that everyone is afraid to delete. `owner: unassigned` is flagged as a **cost accountability** issue for exactly that reason.

Result on the demo estate

Figure	Value
Monthly spend	$7,532
Identified waste	$4,206 (55.8%)
Findings	27 across 10 categories
Identified savings	$4,611/month = **$55,332/year**
Largest single finding	rds-analytics — db.r6g.4xlarge at 9% CPU, $2,715/month

The waste ratio is high on purpose: the inventory was written with realistic failure modes rather than a tidy estate. A mature real environment runs 20–35%.

10. Security

No hardcoded credentials — and what that actually means

There is no credential anywhere in the repository: not a placeholder that works, not a default, not a test fixture. Two properties matter more than the absence itself.

- **Unset means absent, not default.** The webhook sink is only constructed if a URL is present. A misconfigured deployment cannot silently post alerts to somebody else's endpoint because a default was left in the code.
- **An unauthenticated deployment admits it.** `/readyz` and `/api/v1/system` report `auth_enabled: false`. A demo without a token is fine; a demo that *looks* secured while being open is how a laptop deployment gets promoted to a shared environment by mistake.
- `hmac.compare_digest`, not `==`. String equality short-circuits at the first differing byte, so response timing leaks how many leading characters were right and the token can be recovered one character at a time.
- **Split read/write authorisation.** `REQUIRE_TOKEN_FOR_WRITES` keeps GET open — a dashboard on a wall — while incident injection and alert acknowledgement still require the token.
- The Grafana password arrives via `__FILE` and a Docker secret, because environment variables are visible in `docker inspect`.

RBAC and least privilege

- **Every workload has its own ServiceAccount.** Sharing `default` means every pod inherits every permission any one of them needs, and it can never be taken back.
- **No token where none is needed.** The demo workloads never call the API server, so `automountServiceAccountToken: false`. A token that is not mounted cannot be read out of the filesystem by an SSRF or a path-traversal bug.
- **Role, not ClusterRole. Verbs, not wildcards.** Namespaced get/list/watch on pods, services, endpoints, configmaps, deployments, events and pod metrics.
- **Explicitly not granted: secrets.** A monitoring component that can read secrets can read every credential in the namespace. No dashboard is worth that. CI fails the build if `secrets` appears in any rule.
- **Explicitly not granted: create/update/delete/exec.** A read-only identity that can also create pods is a direct escalation path to cluster-admin — create a pod that mounts a privileged service account, done.
- One ClusterRole exists because node objects are cluster-scoped and genuinely cannot be granted with a Role. It reads nodes and node metrics, nothing else.

Network policy: default deny

Kubernetes networking is flat by default — every pod can reach every other pod, so one compromised container can scan the entire estate. The manifests invert this: deny-all both directions, then add back DNS (the exception everyone forgets — without it every pod hangs on name resolution with an error that looks nothing like a policy problem), control plane → workloads on 8080 one-directionally, and scoped ingress.

Egress excludes 169.254.0.0/16. That is the cloud instance metadata endpoint — how a server-side request forgery bug turns into stolen IAM role credentials. Blocking it at the network layer means the application code is not the only thing standing between a bug and a cloud credential.

But verify enforcement. Network policies require a CNI that implements them. On a CNI that ignores them, these objects apply cleanly and do nothing — a genuinely dangerous illusion of security.

Container hardening

Non-root UID 10001, `cap_drop: ALL`, `no-new-privileges`, multi-stage builds. In Kubernetes additionally: `readOnlyRootFilesystem` with explicit emptyDir mounts, `allowPrivilegeEscalation: false`, `seccompProfile: RuntimeDefault`, and the `restricted` Pod Security Standard.

All of this is asserted in CI. A future edit that drops `runAsNonRoot`, adds an RBAC wildcard, grants access to secrets, or commits a literal credential fails the build. Documented policy that is not enforced is a comment.

Application hardening

- All API data inserted with `textContent`, never `innerHTML` — a resource name or log message can never become executable script. No CDN, no inline handlers, no eval.
- Response headers: `nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`.
- Pydantic bounds at the edge: duration 30-3600s, magnitude 0.1-3.0, regex-constrained filters, and a hard maximum `limit` on every list endpoint so no request can materialise the whole retention window.
- Parameterised SQL everywhere; config mounted read-only; secrets redacted to booleans in `/api/v1/system` and verified by the e2e script.

The deliberate exception

`POST /admin/chaos` on the demo services is a genuine remote code-effect endpoint. It exists because inducing real failure is the entire point of the demonstration, and a fake incident would prove nothing about the detection path. It is documented as a deliberate exception rather than left as an accident; the NetworkPolicy already restricts who can reach 8080 to the control plane alone, and in a real deployment it would be compiled out or bound to a separate, unexposed, authenticated port. Nothing analogous exists on the control plane.

11. Startup instructions

Prerequisites

Docker Desktop (or Docker Engine + Compose v2) and Python 3.11+ for the scripts. That is the complete list. No cloud account, no API key, no paid service.

Start it

```
git clone <repo> && cd cloudops-sentinel

make up            # build + start, waits for readiness
make verify        # 50+ check end-to-end verification (~3 min)
make demo          # narrated incident walkthrough

# Dashboard   http://localhost:8000
# API docs    http://localhost:8000/docs
# Metrics     http://localhost:8000/metrics
```

Without make:

```
python -c "import secrets;open('secrets/grafana_admin_password.txt','w')\
  .write(secrets.token_urlsafe(24))"
docker compose up -d --build
curl -s localhost:8000/readyz
python scripts/verify_local.py
```

Every command

Command	Does
make up / down / clean	Start · stop (history kept) · stop and wipe volumes
make verify / verify-quick	Full end-to-end incl. live incident · without the 2-min incident test
make test	69 unit tests
make demo	Narrated incident walkthrough
make observability	Adds Prometheus (:9090) and Grafana (:3000)
make incident SCENARIO=… TARGET=…	Inject one incident
make stop-incidents	Cancel all active incidents
make cost / recommendations / metrics	Cost summary · top findings · Prometheus exposition
make logs / ps	Follow control plane logs · container status
make k8s-validate / k8s-apply	Render + validate manifests · apply to current context

Configuration

Copy `.env.example` to `.env`. Every value has a safe default, so the stack runs correctly with none of them set. Ports 8000/8081-8083/9090/3000 are all overridable. To enable authentication, generate a token with `python -c "import secrets; print(secrets.token_urlsafe(32))"` and set `CLOUDOPS_API_TOKEN`.

First-boot behaviour

On first start the collector backfills six hours of synthetic history (~68,000 samples, about one second) so the dashboard, cost view and anomaly detector are useful immediately rather than showing an empty screen. This is why the container HEALTHCHECK has a 45-second start period and the Kubernetes startupProbe allows 150 seconds. Only simulated resources are backfilled — inventing a past for a container that started thirty seconds ago would put fiction in the same table as measurements.

12. Interview demonstration

A tested seven-minute sequence. Run `make up` beforehand and leave it running for a few minutes so there is history to show.

Minute 0-1: the estate

Open **http://localhost:8000**. Point at the KPI row: 19 resources, fleet health, $7,532/month, 55% waste, $4,611/month identified.

Say: "Nineteen resources across Azure and AWS. Sixteen are simulated from a YAML inventory so I get breadth — RDS, S3, Lambda, load balancers, orphaned disks — without a cloud bill. These three are real Docker containers. Their CPU is read from their own cgroup, the same source the kubelet uses. Everything above the collector treats both identically."

Minute 1-2: cost

Scroll to the cost breakdown. Then in a terminal: `make recommendations`.

Say: "The headline is this RDS instance: sixteen vCPU sitting at 9% CPU, $2,715 a month. The engine proposes a specific size, and it never shrinks memory below the observed peak plus 25% — CPU pressure makes a service slow, memory pressure makes it die. It also refuses to make a sizing recommendation on fewer than eight samples. Confidently telling someone to halve a production database off four data points is how a cost tool loses credibility permanently."

Minute 2-5: the incident (the centrepiece)

In the dashboard's incident panel choose **CPU saturation**, target **checkout-api [live container]**, 180 seconds, Launch. Note the response says `chaos injected into the real container`.

Say: "That just called the container's chaos endpoint. There is now a thread inside it burning CPU against a 0.5-core limit. Nothing told the alert engine an incident was requested — it has to work this out from the numbers."

While waiting, show the container's own view from a second terminal:

```
curl -s localhost:8081/metrics | grep cpu_utilization
```

Then narrate the sequence as it happens on screen:

- **~20s** — CPU climbs past 85% in the resource table.
- **~30s** — health score drops from 100 to 88, reason listed.
- **~60s** — the alert appears as **pending**. *"It will not page yet. The rule has a 60-second `for` duration — one bad scrape is not an incident, and that single mechanism kills most false pages."*
- **~90s** — it transitions to **firing**, critical, with a runbook link.
- **Log stream** — filter to ERROR; the structured alert line is there with rule, value, threshold and runbook.

Click **stop**. *"It resolves on its own once the condition has been clear for 120 seconds — otherwise a flapping target pages you every thirty seconds."*

Minute 5-6: it is real engineering, not a demo script

```
make test                # 69 unit tests
python scripts/verify_local.py --quick
```

Say: "CI runs this same script — there is no CI-only path that can pass while the real thing is broken. And it found real bugs. Three worth mentioning."

1. "The restart alert could never resolve. `restart_count` is a monotonic counter, so `> 2` latches forever. I added windowed increase aggregation — the same reason Prometheus rules are written `increase(x[10m])`."
2. "The anomaly detector went blind after one outlier. My MAD-zero fallback used standard deviation, which reintroduced exactly the non-robustness MAD exists to avoid. A test caught it."
3. "A crash loop that died faster than the scrape interval was never observed at all — a real property of pull-based monitoring."

Minute 6-7: security and Kubernetes

Open `k8s/base/10-rbac.yaml` and the `k8s-validate` CI job side by side.

Say: "Least privilege, and it is enforced rather than documented. The monitoring identity gets get/list/watch in one namespace. It is explicitly not granted secrets — a monitoring component that can read secrets can read every credential in the namespace. CI fails the build if anyone adds it, or adds an RBAC wildcard, or drops runAsNonRoot, or commits a literal credential. And egress excludes 169.254.0.0/16, which is how an SSRF turns into stolen IAM credentials."

Close on limitations, unprompted: *"Sixteen of nineteen resources are simulated, the cost model will not reconcile with an invoice, and SQLite means one writer so the control plane cannot scale horizontally. All of that is in docs/LIMITATIONS.md with the fix for each."* Volunteering this is worth more than being caught by it.

13. The two-minute explanation

For a recruiter, a manager, or the opening "tell me about your project".

"CloudOps Sentinel is a cloud operations platform. It monitors a cloud estate, scores its health, estimates what it costs, detects anomalies, fires alerts, and recommends specific fixes with a dollar figure attached.

The idea behind it is that three questions usually get answered by three different tools that disagree — is it working, will it keep working, and why does it cost this much. But they all come from the same data. A database running at 9% CPU is healthy, and it is also two thousand seven hundred dollars a month of waste. A billing export can never tell you that second part, because the invoice shows cost and never shows waste.

The hard constraint was that it had to run on a laptop with no cloud spend. So the estate is split in half. Sixteen resources are simulated from a YAML inventory — that gives me breadth: RDS, S3, Lambda, load balancers, orphaned disks, two clouds. Three are real Docker containers whose CPU and memory are read from their own cgroup. When I inject an incident into one of those, it genuinely burns CPU or leaks memory or crashes. The detection path is never told an incident was requested — it has to work it out from the measurements, over the network, the way it would in production.

On the demo estate it finds about fifty-five thousand dollars a year in savings across twenty-seven findings, and every one carries the evidence that triggered it and a specific action.

It is Python and FastAPI, containerised, with Kubernetes manifests, Prometheus metrics, a Grafana dashboard and a GitHub Actions pipeline. Sixty-nine unit tests, and a fifty-plus check end-to-end script that CI runs against real containers — including injecting real chaos and asserting the alert fires unaided."

14. The five-minute explanation

For an engineer or a technical panel. Five movements.

1. The problem (30s)

"Three questions get asked about every cloud estate — is it working, will it keep working, why does it cost this much — usually by three different people, answered by three different tools that disagree. But they all derive from the same telemetry. Utilisation is a reliability signal and it is simultaneously the numerator of the cost-efficiency calculation. Splitting them across tools is what lets an organisation argue about whose number is right instead of fixing the problem."

2. The constraint, and the design it forced (60s)

"It had to run on a laptop with no cloud spend. The lazy answer is to mock everything, but then the demo proves nothing — the detection logic is handed the answers. So I split the estate. Sixteen simulated resources give breadth. Three real Docker containers give truth: CPU and memory read from their own cgroup, expressed as a percentage of the container limit — which matters, because a container can be throttled to a standstill on a host that looks idle.

The key property is that everything above the collector treats both identically. The health scorer, the anomaly detector and the alert engine only see a Resource and a stream of samples. They cannot tell which half a number came from."

3. The pipeline (90s)

"Every ten seconds the collector generates samples for the simulated half, scrapes the live containers concurrently, persists both, then runs the analysers.

It is pull, not push, deliberately — a target that dies is detected by its absence. With push, a silent target and a healthy target look identical. And a failed scrape records availability zero rather than skipping the resource, because down is a measurement; if you skip it, 'down' becomes unalertable.

Anomaly detection is a robust z-score — median and median absolute deviation, not mean and standard deviation. That is not pedantry: one enormous outlier inflates a standard deviation enough to hide the next genuine anomaly, so the detector goes blind exactly when it matters. There is a test for it. There is also a second detector for slow ramps, because a memory leak defeats a z-score structurally — the baseline drifts up along with the value.

Alerting follows Prometheus semantics: pending to firing to resolved, with a `for` duration so one bad scrape never pages, stable fingerprints so one problem is one row, and state in SQLite so it survives a redeploy."

4. Cost, and why it is opinionated (60s)

"Efficiency is p95 utilisation over eighty, capped at one — using max of CPU and memory, not the average, because you cannot shrink past the dimension that is full. And over eighty rather than a hundred, because headroom is not waste. If I targeted a hundred percent I would flag every well-run production service as twenty percent wasteful and the report would be dismissed by the people who most need to read it.

Rightsizing snaps to real SKUs, targets sixty percent after the change, never shrinks memory below the observed peak plus twenty-five percent, and refuses to emit anything on fewer than eight samples. It also recommends rightsizing before commitments — buying a one-year reservation for an oversized instance locks in the waste for a year while the invoice shows a saving."

5. Rigour, and honesty about limits (60s)

"Sixty-nine unit tests, a fifty-plus check end-to-end script that CI runs against real containers, and CI that asserts the security posture — it fails the build if an image runs as root, if RBAC gains a wildcard or access to secrets, or if a credential is committed. Documented policy that is not enforced is a comment.

Verification found three real bugs. An alert on a monotonic counter that could never resolve. An anomaly detector that went blind after one outlier because my zero-MAD fallback used standard deviation. And a crash loop that died faster than the scrape interval and was therefore never observed — which is a real property of all pull-based monitoring.

The limits are documented: sixteen of nineteen resources are simulated, the cost model will not reconcile with an invoice — no egress, no enterprise agreements — and SQLite means one writer, so the control plane cannot scale horizontally. Moving to Postgres is the first thing I would do, because it unblocks HA and a thirty-day rightsizing window in one change."

15. Viva questions and answers

Fifty-two questions across architecture, cloud, Docker, Kubernetes, CI/CD, monitoring, logging, cost, security and Python — with the answers that hold up under follow-up.

Architecture and design

Q1. Why join monitoring and cost into one platform instead of using two tools?

Because they derive from the same data and the join is where the value is. Utilisation is a reliability signal and simultaneously the numerator of the cost-efficiency calculation. A database at 9% CPU is healthy, is a capacity question, and is $2,715 a month of waste — all from one number. A billing export can never tell you the third thing, because an invoice shows cost and never shows waste: utilisation lives in the monitoring system and price lives in the billing system. Keeping them separate is also how you get two teams arguing about whose number is right instead of fixing the problem.

Q2. Why simulate most of the estate? Does that not make the whole thing a toy?

The constraint was zero cloud spend, and a real multi-cloud estate producing these findings would cost thousands a month. But the important design decision is that I did not simulate everything. Three resources are real containers with real cgroup measurement and real induced failure, and the crucial property is that everything above the collector treats both halves identically — the health scorer, the anomaly detector and the alert engine only ever see a Resource and a stream of samples.

So the detection path is genuinely exercised. When I push a container to 94% CPU, the alert that fires was not told an incident was requested. The simulated half gives breadth I could not otherwise afford — RDS, S3, Lambda, orphaned disks, two clouds, real SKUs — and the live half gives truth. Swapping in a real cloud is one function: replace load_inventory() with an Azure Resource Graph or AWS Resource Groups Tagging API call.

Q3. Walk me through what happens in one collection tick.

Expire finished incidents and load the active ones. Generate samples for the simulated resources with any incident modifiers applied. Scrape the live containers concurrently with asyncio.gather. Persist samples and logs in two batched writes. Then analyse: score health, run both anomaly detectors, evaluate the alert rules. Publish everything as Prometheus gauges. Prune past the retention window roughly once a minute.

Two details matter. A failed scrape records availability=0 rather than skipping the resource — down is a measurement, and skipping it would make a dead target indistinguishable from a healthy one that simply did not match a rule. And the whole loop is wrapped in a broad exception handler that logs and continues, because a collector that dies on one malformed sample is worse than no monitoring at all: everyone assumes silence means health.

Q4. Why pull rather than push?

One specific property: a target that dies is detected by its absence. With push, a silent target and a healthy target look identical until someone notices the dashboard stopped moving. With pull, the scraper knows immediately that it could not reach the target, and Prometheus even synthesises an `up` metric for it — so you can alert on a target being too broken to emit anything at all, which is exactly when you most need to hear about it.

Pull also puts rate control on the monitoring side rather than the application side, and makes the target list explicit and reviewable rather than emergent.

Q5. Why SQLite? Is that not a toy database?

It is the right tool under the constraint and the wrong tool without it. The requirement was running on a laptop with no managed database. The write rate is a few hundred rows per tick, which SQLite in WAL mode handles comfortably — WAL specifically lets the collector write while the API reads without blocking.

The access pattern is what matters for portability: append-only time series plus small mutable alert and incident tables. That is the same shape you would put in Timescale or Azure Monitor, so the queries port upward without redesign. The real cost is single-writer, and I do not hide it — the control plane is a StatefulSet with one replica because a Deployment with a PVC would let a rolling update run two pods against one SQLite file. That means no horizontal scaling and no HA, which is the first item in my limitations doc, and moving to Postgres is the first thing I would build next because it unblocks HA and a 30-day rightsizing window in one change.

Q6. Why is the control plane a StatefulSet but the demo services are Deployments?

Because the control plane owns a file and the demo services do not. A StatefulSet gives ordering — the old pod is fully terminated before the new one starts — which is what makes single-writer storage safe during a rolling update. A Deployment would happily run both pods concurrently against the same volume and corrupt it.

The converse is the more interesting half: the demo services are stateless, which is exactly why they can have an HPA and the control plane cannot. You cannot horizontally autoscale something that owns a single-writer file. The workload's state model dictates the controller, not the other way round.

Cloud

Q7. Explain the shared responsibility model with an example from this project.

The provider secures of the cloud — hypervisor, physical hosts, managed service internals. You secure in the cloud — identity, network exposure, encryption settings, patching your images, your data.

Every configuration finding this project raises sits on the customer side. The publicly reachable RDS instance: AWS provides a private-subnet option and it is on me not to have used it. The unencrypted OS disk: Azure offers platform-managed encryption and it is off. Anonymous blob access, static credentials instead of a managed identity, diagnostic logs disabled. The provider will let you make every one of those mistakes, and none of them are their fault.

Q8. What is the difference between allocation-billed and consumption-billed resources, and why does it change the recommendation?

Allocation-billed — VMs, managed databases, disks — charge for provisioned capacity whether you use it or not. That is where waste lives and where rightsizing pays: a 16-vCPU database at 9% costs exactly the same as one at 90%.

Consumption-billed — Lambda, Functions, S3 requests — charge for what actually runs. You cannot over-provision them, so the efficiency calculation would be meaningless and my cost model treats them as 100% efficient by definition. Their optimisation levers are different: memory sizing for functions (because you pay GB-seconds, so an over-memoried function wastes money on every invocation) and storage tiering for objects. That is why they get separate analysers rather than being fed through the rightsizing path.

Q9. What is a managed identity or IRSA, and why does the project flag its absence?

It is a workload identity issued and rotated by the cloud platform, so the application authenticates without a secret ever existing. Azure Managed Identity, AWS IAM Roles for Service Accounts. The application asks the platform for a short-lived token; there is no credential to store, rotate, leak, commit, or find in a log.

I flag its absence because static credentials are the single most common root cause of real cloud breaches. A resource with managed_identity: false has a long-lived secret somewhere — in an env var, a config file, a CI secret store — and every one of those is a place it can leak from. The finding is rated high severity, and the fix is not 'rotate it more often', it is 'stop having one'.

Q10. Which Well-Architected pillar does this project cover best, and which worst?

Best is cost optimisation — it is the whole cost engine: waste quantification, rightsizing with headroom, orphan detection, scheduling, tiering and commitment analysis, each with evidence and a dollar figure.

Worst is reliability, and specifically of the platform itself. The control plane is a single replica with single-writer storage and no HA. It monitors reliability well and embodies it poorly. I know exactly why — SQLite — and exactly what the fix is, which is why moving to Postgres is first on my list.

Docker

Q11. Why a multi-stage build?

The builder stage installs dependencies into a virtualenv; the runtime stage copies only that virtualenv and the source. Build tools, pip caches and package indexes never reach the shipped image.

Two benefits. The image is much smaller, so pulls are faster and the layer cache is more effective. And the attack surface shrinks: if someone gets a shell in that container there is no compiler, no package manager and no build tooling to use. It also means a CVE in a build-only dependency is not a CVE in production.

Q12. Why copy requirements.txt before the application source?

Layer caching. Docker invalidates every layer after the first changed one. If I copied the source first, editing any .py file would invalidate the dependency install and reinstall everything on every build. Copying dependencies first means a source edit reuses the cached install layer.

It is the single biggest lever on rebuild time in CI — the difference between a twenty-second build and a three-minute one, on every push.

Q13. Why does your CMD use exec form, and what breaks with shell form?

Signal handling. With exec form, uvicorn becomes PID 1 and receives SIGTERM directly, so the FastAPI lifespan shutdown hook runs, the collector stops cleanly, and the database connection closes.

Shell form leaves /bin/sh as PID 1. sh does not forward signals to its children, so SIGTERM is swallowed, the application never learns it is stopping, and Docker eventually SIGKILLs it after the grace period. In Kubernetes that means every rolling update kills in-flight requests. The demo service does use sh -c because it needs env var expansion for the port, but with `exec` in front so uvicorn still replaces the shell as PID 1 — that is the important detail.

Q14. How do you measure a container's real CPU usage, and why not just read /proc?

From the container's own cgroup — /sys/fs/cgroup/cpu.stat for cumulative usage and cpu.max for the limit — with a fallback chain to cgroup v1 and then /proc/self/stat. That is the same source docker stats and the kubelet use.

The reason /proc alone is wrong is the denominator. /proc tells you the process's CPU time; dividing by the host's core count tells you the percentage of the host, which is useless in a container. What matters is the

percentage of the container's limit, because that is what triggers CFS throttling. A container capped at 0.5 CPU saturating its half-core is at 100% and being throttled, while the host looks completely idle. Getting that denominator wrong is why people say 'the node has plenty of CPU but the service is slow'.

Also, CPU is a rate, so it needs two readings and the elapsed time between them, and I smooth it with an EWMA because a single 200ms window is far too twitchy to alert on.

Q15. Why set resource limits in docker-compose when nothing is competing for resources?

Because the limit is what makes the measurement meaningful. The demo service reports utilisation as a percentage of its own envelope, so a 0.5-CPU limit means saturating half a core reads as 100% — exactly as Kubernetes would compute it against a pod limit. Without a limit the number would be a percentage of my laptop and would mean nothing.

It also makes the chaos demonstration realistic: a CPU burn hits a real ceiling and gets throttled, the way it would in production.

Kubernetes

Q16. Explain liveness, readiness and startup probes, and the most common mistake.

They answer three different questions. Liveness: is the process wedged — if it fails, kill and restart the pod. Readiness: can it serve right now — if it fails, remove it from the Service endpoints but leave it running. Startup: has it finished booting — if it fails, keep waiting.

The most common and most damaging mistake is making liveness depend on a downstream service. If your liveness probe checks the database, then when the database is slow every pod fails liveness simultaneously and Kubernetes restarts the entire fleet — turning a partial outage into a total one, and adding a thundering herd of reconnections to a database that was already struggling. Liveness must only test the process itself.

My demo service demonstrates the correct split: during an induced outage /healthz stays 200 while /readyz returns 503. Restarting is the wrong remedy for a dependency being down; removing from the load balancer is the right one.

The second mistake is using a slack liveness probe to accommodate a slow start. The fix is a startupProbe — it grants a long grace period once, then hands over to a tight liveness probe for the rest of the pod's life. My control plane uses 30 x 5s to cover its first-boot backfill.

Q17. What is the difference between a request and a limit, and what is QoS?

The request is what the scheduler reserves — it decides which node the pod lands on and it is the number the HPA divides by. The limit is the hard ceiling: exceed CPU and you get throttled, exceed memory and you get OOM-killed.

QoS is derived from them. Guaranteed means requests equal limits for every resource, and those pods are evicted last. Burstable means requests are set but lower than limits. BestEffort means neither is set, and those are evicted first under node pressure.

I set memory request equal to limit on the control plane deliberately, so it is Guaranteed — the component that has to still be alive to explain an outage should be the last thing evicted. And the LimitRange gives every container a default, so a pod authored without resources cannot land as BestEffort by accident.

Q18. Why is my HPA not scaling?

In order of likelihood. First, the pods have no CPU request — the HPA computes utilisation as usage divided by request, so with no request there is no denominator and it silently does nothing. That is by far the most common cause. Second, metrics-server is not installed or not returning data; check `kubectl top pods`. Third, it is already at maxReplicas. Fourth, the scale-down stabilisation window is masking what you expect to see — the default is five minutes, deliberately, so a burst that ends does not immediately scale down and start the next burst from cold.

`kubectl describe hpa` tells you which one it is; the events and the current/target column are explicit.

Q19. Why does your rolling update need a preStop hook?

Because endpoint removal and container termination happen concurrently, not in sequence. When a pod is deleted, Kubernetes sends SIGTERM to the container and separately notifies the endpoint controller — which then has to propagate to every kube-proxy and every ingress controller. The container often stops accepting connections before that propagation completes, and traffic is still being routed to it.

A preStop sleep of a few seconds keeps the container serving while the endpoint removal propagates. Omitting it is the single most common cause of 502s during an otherwise correct rolling update — and it looks like a mysterious intermittent failure rather than a configuration problem.

Q20. How does a config change trigger a rollout?

It does not, by default — and that surprises people. A mounted ConfigMap updates the file in the container eventually, but nothing restarts the process, so an application that reads config at startup keeps running the old values indefinitely.

I use the kustomize configMapGenerator, which appends a content hash to the ConfigMap name. Change config/inventory.yaml and the ConfigMap gets a new name, which changes the pod spec, which triggers a rollout automatically. That is the built-in answer to 'I changed the config and nothing happened'. The alternative is a checksum annotation on the pod template, which is what Helm charts typically do.

Q21. Why is your PodDisruptionBudget maxUnavailable: 1 on a single-replica workload? Is that not useless?

It is honest rather than useless. With one replica a PDB genuinely cannot preserve availability — there is no second copy to keep serving. The tempting thing is maxUnavailable: 0, which looks stricter, but all it actually does is make kubectl drain hang forever and block node upgrades. That is a worse operational failure than a twenty-second monitoring gap, and it is the kind of thing that gets discovered at 2am during a cluster upgrade.

So the PDB permits the eviction, and the real remedy is recorded where it belongs — move the store off SQLite and run two replicas. A PDB cannot fix an architecture problem, and pretending it can just hides the problem.

Q22. What does a NetworkPolicy do, and what is the trap?

By default Kubernetes networking is completely flat — every pod can reach every other pod in the cluster, so one compromised container can scan the entire estate. A NetworkPolicy lets you deny by default and then allow specific paths. Mine denies everything both directions, then adds back DNS, control plane to workloads on 8080 one-directionally, and scoped ingress.

Two traps. First, DNS: if you deny egress without explicitly allowing UDP/TCP 53 to kube-dns, every pod hangs on name resolution with an error that looks nothing like a network policy problem. Everyone hits this once.

Second, and more dangerous: NetworkPolicies require a CNI that implements them. On a CNI that ignores them — plain kubenet, or AWS VPC CNI without Calico — the objects apply cleanly, `kubectl get netpol` shows them, and they do absolutely nothing. You have an illusion of security, which is worse than knowing you have none. You have to verify enforcement with an actual deny test.

CI/CD

Q23. Why is your pipeline ordered the way it is?

By how fast a stage can tell you that you are wrong. Lint and unit tests take seconds and run first. The container build takes about a minute. The end-to-end stage brings the whole stack up and takes a few minutes. Nothing downstream runs if something upstream failed.

The economics are about developer attention, not CPU time: a failure surfaced in thirty seconds gets fixed immediately, while a failure surfaced after eight minutes gets discovered after the developer has context-switched to something else, and costs far more than the eight minutes.

Q24. Why does secret scanning fail the build but a base-image CVE does not?

Because one is always actionable and the other frequently is not. A committed credential has exactly one correct response — revoke and remove — and it is always available. So it is a hard failure.

A CRITICAL CVE in a base image often has no fixed version yet. If that fails the build, the pipeline is red for days through no fault of any developer, and people learn to click through red builds. Then when a real failure appears, they click through that too. An unactionable alert does not just waste attention, it actively destroys the value of the actionable ones. So base-image findings go to the security tab as SARIF where they can be triaged, and the build stays meaningful.

Q25. How do you handle credentials in the pipeline?

GITHUB_TOKEN with job-scoped permissions rather than a long-lived personal access token stored as a secret. Default permissions are contents: read at the workflow level, and each job opts into more — only the publish job gets packages: write. There is nothing to rotate and nothing to leak.

Publishing runs only on a push to main and never on a pull request, so a fork PR cannot push an image — that is a real and commonly exploited attack path. For cloud deployment the pattern is OIDC federation: azure/login with a federated credential, or configure-aws-credentials with role-to-assume. No static cloud key ever sits in a repository secret.

Q26. What stops someone regressing your security posture in six months?

The k8s-validate job. It renders the manifests with kustomize and then runs assertions on the rendered YAML: runAsNonRoot must be true, no workload may use the default ServiceAccount, allowPrivilegeEscalation must be false, root filesystem read-only, all capabilities dropped, requests and limits present, a readiness probe present, no wildcard verbs in any RBAC rule, no rule granting secrets, no literal value on any env var whose name looks like a credential, and the namespace must enforce the restricted Pod Security Standard.

That is the difference between a documented policy and an enforced one. A SECURITY.md that nothing checks is a comment. If someone drops runAsNonRoot to debug something and forgets to put it back, the build fails and tells them which workload.

Q27. Why does CI run the same script a developer runs?

So there is no CI-only test path that can pass while the real thing is broken. If CI had its own bespoke health check, it could drift from what actually matters — and the classic failure is a green pipeline shipping something a developer could see was broken in thirty seconds locally.

scripts/verify_local.py boots the stack, injects real chaos into a real container, and asserts the detection path noticed. `make verify` and the CI job are the same command.

Monitoring, logging and detection

Q28. Why median and MAD instead of mean and standard deviation?

Robustness. A single enormous outlier drags the mean toward it and inflates the standard deviation enormously — so the threshold widens and the next genuine anomaly falls inside it. The detector goes blind exactly when something has just gone badly wrong, which is the worst possible time.

Median and median absolute deviation are unaffected by a minority of extreme values. I have a test for it: with a history of fifty-nine 40s and one 5000, a current value of 95 is still detected. The 1.4826 scaling factor makes MAD a consistent estimator of sigma for normally distributed data, so the z-score keeps its usual interpretation.

There is a subtlety I got wrong first time. When the series is perfectly flat, MAD is zero and any z-score explodes. My first fix fell back to standard deviation — which reintroduced exactly the non-robustness MAD exists to avoid, and a test caught it. The correct fallback is the metric's domain-significance floor: on a flat series, 'meaningfully different' is defined by what a human would call meaningful for that metric, not by the noise.

Q29. Why not use machine learning for anomaly detection?

Three reasons, and I would revisit all of them at a different scale. Explainability: I can tell an on-call engineer 'this is 6.2 median absolute deviations above the last ten minutes' and they can act on it. 'The model said so' does not survive a 3am page. Cold start: this works on the twelfth sample of a brand-new resource, with no training job, no model artefact and no feature store. And operational cost: there is no retraining pipeline to own, no drift to monitor, no second system that can fail silently.

Where ML would genuinely earn its place is seasonality — a legitimate Monday morning ramp currently registers as drift. Seasonal decomposition or Prophet would fix that, and it is in my limitations doc.

Q30. What is wrong with alerting on restart_count > 2?

It can never resolve. restart_count is a monotonic counter, so once it passes two it stays above two forever, and the alert latches permanently. You end up with a critical alert that is always firing, which people mute, and then they mute the whole class.

The fix is to alert on growth over a window rather than the absolute value — increase(restart_count[10m]) > 2 in PromQL, and I implemented the equivalent as a windowed 'increase' aggregation in my rule engine. It fires when the container is actually crashing now and resolves when it stops.

This was a real bug in my own code, caught during verification. The general rule is: never alert on a raw counter, always on a rate or an increase.

Q31. What is a `for` duration and why does it matter?

The condition must hold continuously for that long before the alert fires. My CPU rule is 60 seconds, so a single scrape catching a transient spike moves the alert to 'pending' and it only becomes 'firing' if it is still true a minute later.

This single mechanism eliminates the majority of false pages. Real systems are noisy — one GC pause, one slow scrape, one brief burst — and none of those are incidents. Without a `for` duration you page on every transient, people stop trusting the pager, and then they miss the real one. It is the cheapest possible improvement to alert quality.

The symmetric mechanism matters too: I require the condition to be clear for 120 seconds before resolving, so a flapping target does not generate a fresh page every thirty seconds.

Q32. Why is your dashboard one aggregate endpoint instead of several?

The dashboard polls every five seconds. Eight endpoints per refresh means eight requests, eight round trips, eight sets of overlapping database queries — and the same underlying data fetched repeatedly. One /api/v1/overview call keeps the browser simple and bounds the server's work per refresh to one pass over the data.

The trade-off is that the payload is larger and you cannot refresh one panel independently. At this scale that is clearly the right side of the trade. At a thousand resources I would paginate the resource table and split it.

Q33. Why do you log JSON to stdout instead of writing log files?

It is the twelve-factor contract, and it makes the application portable across every log pipeline without knowing which one it is running under. The container writes to stdout, the runtime captures it, and something downstream forwards it — Docker's json-file driver, a Fluent Bit DaemonSet, Container Insights, CloudWatch, Promtail.

Writing log files inside a container creates three problems at once: the files vanish when the container is replaced, they fill the writable layer, and they are invisible to platform log collection. My Kubernetes manifests mount a read-only root filesystem, which makes it impossible by construction rather than by convention.

JSON specifically because structured records are queryable immediately — `| json | severity="critical"` works whatever the message text says. A formatted string needs a regex that breaks the moment someone rewords the message.

Q34. What is label cardinality and why does it matter here?

Every unique combination of label values creates a separate time series in Prometheus, and memory scales with the number of series. Cardinality is multiplicative across labels, so an unbounded label means unbounded series and eventually an OOM.

The classic mistake is labelling HTTP metrics by raw path. /inventory/azure-vm-web-01 and /inventory/aws-rds-analytics become different series, so you get one series per resource — and with user IDs or request IDs in the path, unbounded. I label by route template instead: /api/v1/inventory/{resource_id} is one label value. My fleet metrics are bounded by the inventory size, a few dozen series.

Q35. How does your platform know it is broken itself?

A CollectorStalled alert on `time() - cloudops_last_collection_timestamp_seconds > 120`, plus a counter of failed collection ticks and a histogram of tick duration.

This matters more than it sounds. The failure mode of a monitoring system is silence, and silence is indistinguishable from everything being fine — every dashboard keeps showing the last values it had, and stale data looks exactly like healthy data. Without a self-check, the way you find out your monitoring died is that an incident goes unnoticed.

The honest limitation is that this alert is evaluated by the thing it is monitoring. A truly robust setup needs a dead-man's switch in an external system — a heartbeat that pages when it stops arriving.

Cost optimisation

Q36. How do you calculate waste?

Two numbers per resource. Provisioned cost is what the rate card charges for the allocated capacity. Effective cost is what that capacity would cost sized to observed p95 utilisation. Waste is the difference.

Efficiency is min(max(cpu_p95, memory_p95) / 80, 1). Two deliberate choices there. Max rather than average, because a box at 80% memory and 5% CPU is not 42% efficient — you cannot shrink past the dimension that is actually full, and averaging would recommend halving a box that would immediately start OOM-killing. And divide by 80 rather than 100, because headroom is not waste: a resource at 80% is correctly sized, and if I targeted 100% I would flag every well-run production service as 20% wasteful and the report would be dismissed by the people who most need to read it.

Q37. How do you avoid a rightsizing recommendation causing an incident?

Four guards. Snap to a real SKU — '6.4 vCPU' is useless because no cloud sells it, so I round down a ladder of purchasable sizes. Target 60% utilisation after the change, not 100%, so there is headroom for a spike or a failover. Never shrink memory below the observed peak plus 25%, because CPU pressure makes a service slow while memory pressure makes it die — they are not symmetric risks and I do not treat them as such. And refuse to emit anything on fewer than eight samples.

The recommendation text also says to change non-prod first and watch p95 latency for a full traffic cycle. And I mark risk as medium on resizes and high on decommissioning, because saving money does not make an irreversible delete low-risk.

Q38. Why recommend rightsizing before a reserved instance?

Because a commitment locks in whatever size you buy, for one or three years. If you buy a reservation for an oversized instance you have locked in the waste — and worse, the invoice now shows a saving, so it looks like you optimised. You have made the problem permanent and invisible at the same time.

My commitment analyser skips anything below 40% p95 CPU for exactly that reason, and its action text says 'rightsize first, then commit to the resulting size'. The order matters more than either action individually.

Q39. What is your cost model missing?

A lot, and I would not quote a number from it to a finance team. No Enterprise Agreements, negotiated discounts, existing reservations or credits. No data egress, which is frequently 10-20% of a real bill and the single most commonly underestimated line item. No inter-AZ transfer, no NAT gateway processing, no snapshot storage, no provisioned IOPS, no support plan percentage, no tax. Spot is modelled as a flat 70% discount with no interruption rate, so it is an upper bound.

It is directionally right and precisely wrong, and it is built to make the optimisation logic demonstrable offline. For real numbers you use the Azure Retail Prices API or the AWS Price List API for rates, and Cost Management or the Cost and Usage Report for actuals. The analyser logic does not change — only the rate card does.

Q40. Your rightsizing window is 24 hours. Is that enough?

No, and it is in my limitations doc. Twenty-four hours misses weekly cycles, month-end batch runs and seasonal peaks. It would happily recommend shrinking a box that is only busy on the last day of the month, and that recommendation would cause an incident thirty days later — long after anyone would connect the two.

Real rightsizing needs 14 to 30 days. The blocker is not the analyser, it is the store: SQLite with a 24-hour retention. Moving to Postgres or Timescale unblocks the longer window and HA in the same change, which is why it is first on my list.

Security

Q41. How do you handle secrets, and what makes 'no hardcoded credentials' more than a slogan?

Every secret comes from an environment variable with no default value, and two properties make that meaningful rather than cosmetic.

First, unset means absent rather than default. The webhook sink is only constructed if a URL is present — there is no fallback endpoint anywhere in the code, so a misconfigured deployment cannot silently post alerts to somebody else's server.

Second, an unauthenticated deployment says so. /readyz and /api/v1/system report auth_enabled: false. A demo without a token is fine; a demo that looks secured while being open is how a laptop deployment gets promoted to a shared environment by mistake.

The Kubernetes Secret template ships with empty stringData deliberately, so applying it creates an empty secret rather than a working credential shared by everyone who cloned the repo. And CI fails the build on any committed secret. In production I would remove the secret from Kubernetes entirely — Workload Identity with Key Vault, or IRSA with Secrets Manager — because a secret that is never written into etcd cannot leak out of etcd. Kubernetes Secrets are base64, not encryption.

Q42. Why hmac.compare_digest instead of ==?

Timing attacks. String equality short-circuits at the first differing byte, so comparing a wrong token takes measurably less time than comparing one that shares a prefix. An attacker who can measure response times recovers the token one character at a time, which turns an infeasible brute force into a few thousand requests.

compare_digest takes the same time regardless of where the difference is. It is one function call and it closes the whole class.

Q43. Explain least privilege in your Kubernetes RBAC.

Every workload gets its own ServiceAccount — sharing `default` means every pod inherits every permission any one of them needs, and you can never take it back. The demo workloads have automountServiceAccountToken: false because they never call the API server, and a token that is not mounted cannot be stolen out of the filesystem by an SSRF or a path-traversal bug.

The control plane gets a namespaced Role with get, list and watch — not a ClusterRole, not wildcards. Two exclusions matter most. No `secrets`: a monitoring component that can read secrets can read every credential in the namespace, and no dashboard is worth that. And no create, update, delete or pods/exec: a read-only identity that can also create pods is a direct escalation path to cluster-admin, because you create a pod that mounts a privileged service account.

There is exactly one ClusterRole, because node objects are cluster-scoped and genuinely cannot be granted with a Role. It reads nodes and node metrics and nothing else. CI asserts all of this.

Q44. Why does your egress policy exclude 169.254.0.0/16?

That is the link-local range containing the cloud instance metadata endpoint, 169.254.169.254. It is how a server-side request forgery bug becomes stolen cloud credentials: an attacker gets your application to fetch that URL and it returns the IAM role credentials attached to the instance. It has caused several very large real-world breaches.

Blocking it at the network layer means the application code is not the only thing standing between a bug and a cloud credential. It is defence in depth — my app does not need metadata access, so it should not have it, and then an SSRF is just a failed request.

Q45. You have an unauthenticated endpoint that makes containers crash. Defend that.

It is a deliberate exception and I document it as one rather than leaving it as an accident. Inducing genuine failure is the entire point of the demonstration — a fake incident would prove nothing about the detection path, which is the thing I actually want to show works.

It is mitigated: the NetworkPolicy restricts reachability on port 8080 to the control plane alone, so it is not reachable from elsewhere in the cluster. In a real deployment it would be compiled out at build time, or bound to a separate admin port that is not exposed outside the cluster and sits behind authentication.

The important part is that nothing analogous exists on the control plane. Its only state-changing endpoints are incident orchestration and alert acknowledgement, and both sit behind REQUIRE_TOKEN_FOR_WRITES.

Q46. How do you prevent XSS in the dashboard?

Every value that comes from the API is inserted with textContent, never innerHTML. textContent does not parse HTML, so a resource name or a log message containing a script tag renders as literal text rather than executing. That matters here because log messages can contain attacker-influenced content — a request path, an error string from a dependency.

There is also no CDN, no inline event handlers and no eval, so the page is compatible with a strict Content-Security-Policy, and the responses carry nosniff, X-Frame-Options: DENY and Referrer-Policy: no-referrer.

Python and implementation

Q47. Why is the correlation ID in a ContextVar rather than a global?

Because this is async. FastAPI serves many requests concurrently on one thread, interleaving at every await point. A module-level variable would be overwritten by whichever request most recently set it, so one request's correlation ID would appear in another request's log lines — and the bug would be intermittent, load-dependent and nearly impossible to reproduce.

ContextVar is the async-safe equivalent of thread-local storage: each task gets its own view, and the value follows the task across awaits.

Q48. Your simulator is deterministic. Why does that matter?

value(resource, metric, t) is a pure function of its arguments and the seed — no RNG state — so any sample can be recomputed at any time. Three consequences.

The six-hour backfill on first boot is continuous with the live samples collected a moment later: no seam, because both come from the same function. The demo reproduces exactly, which matters when you are presenting. And restarting the service does not re-randomise history.

The second design point is that it is not white noise. I use value noise — hash-anchored samples interpolated with smoothstep, summed over two octaves — plus a diurnal sine. Real infrastructure wanders, and that autocorrelation is what makes a z-score detector meaningful. With independent random draws every detector looks brilliant for entirely the wrong reason, and I have a test asserting the series is autocorrelated.

Q49. How does the collector survive a target that is down?

The scrape is wrapped in a try/except for HTTP and OS errors. On failure it records availability=0 and error_rate=1 rather than skipping the resource, increments a scrape-failure counter, and logs a warning with the endpoint and the error.

Recording rather than skipping is the important part — down is a measurement, and the TargetDown alert rule matches on availability < 1. If I skipped, the resource would simply have no recent sample and would silently look the same as one nobody had asked about.

The scrapes also run concurrently under asyncio.gather with return_exceptions=True, so one hanging target cannot stall the others or the tick.

Q50. Why is the log buffer a bounded deque?

maxlen=500 is load-bearing. An unbounded buffer in a container with a 512 MiB memory limit grows until the container is OOM-killed — and it would happen on a busy afternoon, so it would look like an application bug or a traffic problem rather than a logging bug.

A deque with maxlen evicts the oldest record automatically in O(1), so memory is bounded by construction. Logs are still written to stdout unconditionally, which is the real durability path; the buffer is only a convenience so the collector can pull them without a log aggregator deployed.

Q51. What happens if an alert sink raises an exception?

It is caught, logged with the sink name, and the loop continues. A dead webhook must never take down the collection cycle that produced the alert — otherwise one misconfigured integration stops all monitoring, which is a spectacular way to turn a small problem into a large one.

There is a test for it: an engine with a sink that always raises still transitions the alert to firing and persists it correctly.

Q52. If you had another week, what would you build?

In order. First, move the store to Postgres or Timescale — it unblocks HA, horizontal scaling and a 30-day rightsizing window in one change, and everything else is downstream of it. Second, a real cloud connector behind the existing Resource interface: Azure Resource Graph and AWS Resource Groups Tagging API. Third, Alertmanager integration for routing, inhibition and maintenance windows — my webhook payload is already Alertmanager-shaped, and inhibition specifically, because right now a TargetDown does not suppress the HighErrorRate alert for the same resource and one incident produces several correlated pages. Fourth, a kind cluster in CI to turn the Kubernetes manifests from validated into verified. Fifth, seasonality-aware detection so Monday morning is not an anomaly.

16. Troubleshooting questions

These are asked as scenarios — the interviewer wants the diagnostic order, not the answer. Say what you would check *first* and why.

T1. The dashboard shows no data. Walk me through it.

Outside in. Is the container running — `docker compose ps`. Is it ready — `curl localhost:8000/readyz`, which reports which specific check failed: store, inventory, or collector. Is the collector ticking — `/api/v1/system` shows last_collection and the sample count, and the log lines say samples_written each tick.

If readiness says the inventory check failed, the config mount is wrong or the YAML is malformed — the service refuses to start with a bad inventory rather than starting empty, which is the correct loud failure. If the collector has never ticked, check the logs for a traceback during backfill. If everything is green but the browser is empty, it is the browser: check the console and whether /api/v1/overview returns 200.

T2. A container is in CrashLoopBackOff. What do you do?

`kubectl describe pod` first, for the Events — they say why: OOMKilled, ImagePullBackOff, failed probe, FailedScheduling. Then `kubectl logs --previous`, and the --previous flag is the important part: without it you get the logs of the container currently starting, not the one that died.

Then work the exit code. 137 is SIGKILL, almost always OOM — check lastState.terminated.reason and raise the memory limit, but confirm it is not a leak first, because raising the limit on a real leak just buys time. 143 is SIGTERM, usually a failing liveness probe. 1 is an application error, so read the logs. 0 means the process exited cleanly, which means a wrong command or a job deployed as a Deployment.

The most common false crash loop is a slow-starting app killed by an aggressive liveness probe. The fix is a startupProbe, not a slacker liveness probe.

T3. Latency is up but CPU and error rate are normal. Where do you look?

That combination points away from the service itself. If latency is up while throughput is flat and CPU is flat, the time is being spent waiting — a slow downstream dependency, database, cache or external API.

I would check dependency latency for the same window, then connection pool saturation, because a pool that is too small looks exactly like a slow database: requests queue for a connection while the database itself is idle. Then I would check whether timeouts exist on every outbound call, because a missing timeout turns one slow dependency into thread-pool exhaustion across the whole service.

The table in my runbook covers the four combinations: latency up with throughput up is load, latency up with throughput flat is a dependency, latency up with throughput down is saturation, and latency up with CPU up but throughput flat is GC pressure or CPU throttling.

T4. An alert has been firing for three days and nobody has acted on it. What is wrong?

Either the alert is not actionable or the threshold is wrong — and in both cases the alert is the problem, not the people.

First question: does it link to a runbook that says what to do? If not, that is the fix. My rule loader requires a runbook field and CI fails the build without one, for exactly this reason.

Second: can it resolve at all? A rule comparing a monotonic counter latches forever — I had this bug with restart_count and fixed it with windowed increase aggregation. Third: is the threshold set where a human genuinely needs to act, or where it looked reasonable in a config file? An alert that fires when nothing needs doing trains people to ignore the whole class, and then they miss the real one.

T5. Prometheus shows the target as down but curl works from your laptop. Why?

Something between Prometheus and the target that is not between your laptop and the target. Most likely a NetworkPolicy — if Prometheus is in a different namespace, my policy only allows ingress from namespaces labelled monitoring or ingress-nginx.

Then check the obvious ones: is Prometheus resolving the right address (service DNS inside the cluster, not localhost), is the Service selector actually matching pods (`kubectl get endpoints` — empty means readiness is failing or the labels do not match), and is the scrape path and port right in the scrape config. `kubectl get endpoints` is the fastest single check because it distinguishes a networking problem from a 'no ready pods' problem immediately.

T6. Your cost numbers do not match the actual bill. Explain.

They will not, and I say so before anyone asks. My rate card is a simplified list-price approximation. It has no Enterprise Agreement, no negotiated discount, no existing reservations, no credits, and critically no data egress — which is often 10-20% of a real bill and the most commonly underestimated line.

The model exists to make the optimisation logic demonstrable offline, not to reconcile with an invoice. The relative signal — which resources are wasteful and by roughly how much — is sound, because it is driven by measured utilisation. The absolute number is not. For real figures you use the Cost Management API or the Cost and Usage Report, and the analyser logic does not change; only the rate card does.

T7. You deployed a config change and nothing happened. Why?

Because a mounted ConfigMap update does not restart anything. The file in the container updates eventually, but the process already read its config at startup and keeps running the old values indefinitely. This surprises people because the ConfigMap in the cluster clearly shows the new value.

My kustomize configMapGenerator appends a content hash to the name, so a config change produces a new ConfigMap name, which changes the pod spec, which triggers a rollout automatically. The alternative is a checksum annotation on the pod template. Either way the principle is that the config has to be part of the pod spec for a change to be a deployment.

T8. The unit tests pass but the deployment is broken. How would you have caught it?

That is precisely why the end-to-end script exists and why CI runs the same one a developer runs. Unit tests cover the analysis engines in isolation; they cannot catch a wrong mount path, a missing environment variable, a container that will not start as non-root, or a probe pointing at the wrong port.

verify_local.py boots the real stack and asserts behaviour end to end — including injecting real chaos and checking the alert fires. It caught three real bugs that unit tests could not have: the duplicated access log, a crash loop that died faster than the scrape interval, and a resource being scored 'unknown' when it legitimately has no metrics.

17. Deployment questions

D1. How would you deploy this to a real Kubernetes cluster?

`kubectl apply -k .` from the repository root, which renders the namespace with the restricted Pod Security profile, RBAC, ConfigMaps generated from config/, the control plane StatefulSet with its PVC, the workload Deployments, the HPA and the NetworkPolicies.

Before that, three things have to be real: a StorageClass that can bind the PVC, metrics-server for the HPA, and a CNI that actually enforces NetworkPolicy. And the Secret must be created out of band — the committed template is deliberately empty so applying it cannot create a working shared credential.

Then `kubectl rollout status statefulset/control-plane --timeout=180s` and run verify_local.py against the ingress. That last step is what makes it a deployment rather than an apply.

D2. What is your rollback strategy?

`kubectl rollout undo deployment/` for the stateless services, which reverts to the previous ReplicaSet — that is why I keep revisionHistoryLimit: 3. Combined with maxUnavailable: 0, the rollback is itself zero-downtime.

The control plane is harder because it is stateful. The image can be rolled back the same way, but a schema change cannot, so migrations have to be backwards-compatible — expand-then-contract: add the new column, deploy code that writes both, backfill, deploy code that reads the new one, then drop the old column in a later release. That way any single step is reversible.

In CI the deploy stage would run the verification script after the rollout and trigger the undo automatically on failure.

D3. How do you achieve zero-downtime deployments?

Four things together, and all four are necessary. maxUnavailable: 0 with maxSurge: 1, so capacity never drops. Readiness probes, so a new pod receives traffic only when it can actually serve. A preStop hook, so the old pod keeps serving while endpoint removal propagates to every kube-proxy — without it you get 502s during an otherwise correct rollout. And graceful SIGTERM handling in the application, which needs exec-form CMD so the process is PID 1 and actually receives the signal.

Miss any one and you drop requests. The preStop and the exec form are the two people most often miss, because everything looks correct without them.

D4. How would you scale this to 10,000 resources?

Several things break at once, in a predictable order.

First the store: SQLite has to become Postgres or Timescale, and the metric series should go to Prometheus or Mimir rather than rows in my own database. Second the collector: a single sequential tick will not finish in ten seconds, so it needs sharding by resource with a consistent hash, and multiple collector replicas — which is only possible once the store is not single-writer. Third the API: /api/v1/overview returns every resource, so it needs pagination and server-side filtering, and the dashboard needs virtualised rows. Fourth the analysers: the recommendation engine currently runs over the whole estate per request; it should be a periodic job writing results to a table that the API reads.

Cardinality also becomes the binding constraint on the Prometheus side — 10,000 resources times nine metrics is 90,000 series before any other labels, which is fine, but it stops being fine if anyone adds a high-cardinality label.

D5. How do you handle database migrations for the control plane?

Today the schema is created idempotently with CREATE TABLE IF NOT EXISTS at startup, which is honest for SQLite and inadequate for anything real.

For Postgres I would use Alembic with migrations run as an init container or a Job that must complete before the new pods start — not in the application's startup path, because with multiple replicas you get concurrent migrations racing each other. And every migration expand-then-contract, so the old and new code can both run against the intermediate schema. That is what makes a rollback possible at all: if a migration is destructive, rolling back the image does not roll back the data.

D6. What would you monitor about the deployment itself?

Deployment frequency, lead time, change failure rate and time to restore — the four DORA metrics, because they measure the delivery system rather than individual events.

Operationally: rollout duration and whether it completed, pod restart count in the hour after a deploy, and error rate and p95 latency compared against the pre-deploy baseline. That last comparison is what turns 'the deploy succeeded' into 'the deploy was good', and they are not the same statement — a rollout can complete perfectly while the new version is failing 5% of requests.

CI already emits a job summary with the commit and image digest, so a running image traces back to the exact workflow run that produced it.

D7. How would you make the control plane highly available?

The store is the whole problem, so it is the whole answer. Move the mutable tables to managed Postgres with a replica, and the time series to Prometheus or Mimir. Then the control plane becomes stateless and can be a Deployment with two or more replicas across zones, behind a Service, with a meaningful PodDisruptionBudget.

One subtlety: the collector must not run in every replica or every target gets scraped N times and alert evaluation races. Either elect a leader with a Lease — the standard Kubernetes pattern, and client-go has it built in — or split the collector into its own single-replica workload and let the API scale independently. I would prefer the split, because it lets the read path scale for dashboard load without touching the write path at all.

18. Resume bullets

Pick four to six. Each pairs a concrete action with a measurable outcome, and every number is one the project actually produces.

Full version

- Built **CloudOps Sentinel**, a cloud monitoring, reliability and cost-optimisation platform in **Python/FastAPI** covering a 19-resource Azure/AWS estate, identifying **$55K/year in savings across 27 findings** — each with supporting evidence, a specific remediation and a dollar figure.
- Designed a rule-based **recommendation engine** with 10 analysers (rightsizing, idle and orphaned resources, storage tiering, non-prod scheduling, commitment discounts, health and posture) that quantifies **55% of estate spend as unused capacity** and refuses to emit sizing advice on insufficient evidence.
- Implemented **anomaly detection** using robust statistics (median + MAD) plus a trend detector for slow ramps, chosen over mean/stdev because a single outlier inflates variance enough to mask subsequent anomalies — verified by regression tests.
- Built a **Prometheus-compatible metrics pipeline** and alerting engine with pending→firing→resolved lifecycle, *for* durations, windowed counter aggregation and persistent alert state, exposing 20+ metric families consumed by a provisioned **Grafana** dashboard.
- Containerised with **multi-stage Docker builds** (non-root UID, all capabilities dropped, no-new-privileges, health checks) and authored production-grade **Kubernetes** manifests: StatefulSet/Deployments, HPA, PDBs, ResourceQuota, LimitRange, default-deny NetworkPolicies and least-privilege RBAC under the restricted Pod Security Standard.
- Built a **7-stage GitHub Actions pipeline** (lint → test → build → e2e → security → publish → gated deploy) that boots the full stack, **injects real chaos into live containers and asserts the detection path reacts unaided**, and fails the build on root containers, RBAC wildcards, secrets access or committed credentials.
- Engineered a **realistic incident simulation mode** — 9 scenarios inducing genuine CPU saturation, memory leaks, crash loops and 5xx bursts in live containers via cgroup-measured chaos injection, with time-boxed automatic recovery.
- Achieved **69 unit tests and a 50+ check end-to-end verification suite** run identically by developers and CI; verification surfaced three latent defects including an alert on a monotonic counter that could never resolve.

Compact version (three lines)

- Built a cloud monitoring and FinOps platform (**Python, FastAPI, Docker, Kubernetes, Prometheus, Grafana**) over a 19-resource Azure/AWS estate, surfacing **$55K/year of savings** and 55% waste with evidence-backed, actionable recommendations.
- Engineered the full observability pipeline — structured JSON logging, Prometheus metrics, robust-statistics anomaly detection and Prometheus-semantics alerting with runbook-linked rules — plus a chaos-injection mode that induces **real** container failures to validate detection end to end.
- Shipped it with a **7-stage CI/CD pipeline** that verifies the live deployment and enforces security posture (non-root containers, least-privilege RBAC, default-deny networking, zero committed credentials) as build-breaking assertions.

Skills this evidences

Area	Demonstrated by
Azure / AWS	Multi-cloud inventory, service equivalents, pricing models, managed identity/IRSA, Well-Architected pillars
Docker	Multi-stage builds, non-root, capability dropping, health checks, cgroup measurement, Compose with anchors, secrets and profiles
Kubernetes	StatefulSet vs Deployment, three probe types, requests/limits/QoS, HPA, PDB, quotas, NetworkPolicy, RBAC, PSA, kustomize
CI/CD	7-stage pipeline, layer caching, artifact passing, SARIF, OIDC federation, gated environments, policy-as-CI-assertion
Linux	cgroup v1/v2, /proc, signals and PID 1, process lifecycle, exit codes, file descriptors
Monitoring	Golden signals, pull vs push, cardinality, recording rules, SLOs, multi-window burn rate, self-monitoring
Logging	Structured JSON, correlation IDs, ContextVar, twelve-factor stdout, level discipline, bounded buffers
REST APIs	Versioning, resource modelling, bounded pagination, correct status codes, OpenAPI, aggregate endpoints
Python	asyncio, FastAPI, Pydantic, dataclasses, ContextVar, robust statistics, pytest with fixtures
Security	Zero hardcoded credentials, constant-time comparison, least privilege, threat modelling, supply chain, XSS prevention
Cost optimisation	Provisioned vs effective spend, waste quantification, safe rightsizing, commitments, tiering, tagging governance

19. Limitations

Kept deliberately and stated first, not extracted under questioning. A project that claims no limitations is either untested or misrepresented, and the fastest way to fail a technical interview is to defend a weakness you have not noticed.

#	Limitation	Why	Fix
1	16 of 19 resources are simulated	Zero cloud spend was the hard constraint	Replace load_inventory() with Resource Graph / Resource Groups Tagging API — nothing downstream changes
2	Cost model will not reconcile with an invoice	No EA, discounts, reservations, credits, egress, inter-AZ, NAT, snapshots, IOPS, support, tax	Retail Prices / Price List API for rates; Cost Management / CUR for actuals
3	24-hour rightsizing window	Bounded by SQLite retention	14-30 days, after moving the store
4	SQLite means one writer	Must run on a laptop	Postgres/Timescale — unblocks HA and horizontal scale together
5	No downsampling; coarse pruning	Simplicity	Continuous aggregates or recording rules + a long-retention tier
6	Restart detection is inferred	The authoritative source needs the Docker socket, which is root on the host	Read containerStatuses[].restartCount via the existing read-only Role
7	No seasonality in detection	Explainability and cold-start safety chosen over sophistication	STL decomposition or Prophet
8	Minimal alert routing; no inhibition	Scope	Alertmanager — the webhook payload is already Alertmanager-shaped
9	Dashboard polls; no pagination	Simplicity at 19 resources	SSE, server-side pagination, virtualised rows
10	K8s manifests validated, not deployed	A real cluster costs money; a kind cluster in CI tests kind	kind job in CI, plus a staging cluster
11	Unauthenticated chaos endpoint	Real failure injection is the point of the demo	Compile out, or a separate authenticated admin port
12	Single-tenant, no user model	Scope	OIDC, RBAC mapped to groups, per-user audit
13	No load, soak or browser testing	Scope	Behaviour at 10,000 resources is genuinely unknown
14	Only the cgroup v2 path is exercised	Developed on Docker Desktop	Test the v1 and /proc fallbacks on other hosts

What I would build next, in order

1. **Postgres/Timescale store** — unblocks HA, horizontal scale and a 30-day rightsizing window in one change. Everything else is downstream of it.
2. **Real cloud connector** — Azure Resource Graph and AWS Resource Groups Tagging API behind the existing Resource interface.
3. **Alertmanager integration** — routing, inhibition and maintenance windows. Inhibition especially: a TargetDown should suppress the HighErrorRate alert for the same resource, and today it does not.
4. **kind cluster in CI** — turn the Kubernetes manifests from validated into verified.

5 **Seasonality-aware detection** — so a Monday morning traffic ramp is not an anomaly.

Closing note

The strongest thing to say about this project is not what it does — it is that the verification suite found three real bugs in it, and each one is a mistake with a general lesson: never alert on a raw counter, never let a robustness fallback reintroduce the non-robustness it was protecting against, and pull-based monitoring cannot see a process that dies faster than the scrape interval. Being able to name your own bugs and what they taught you is worth more in an interview than any feature list.